

FitBuddy AI

3. Project Design Phase

Architecture, data flow, UI structure and module responsibilities.

High-Level Architecture

- Browser / Jinja2 UI → FastAPI route → input validation → prompt construction → Gemini service → response processing → Jinja2 result page.

Recommended Project Structure

- app/main.py — FastAPI application entry point.
- app/routes/ — application routes/endpoints.
- app/services/ — Gemini/prompt/business logic.
- app/models/ — request/response models when used.
- templates/ — Jinja2 HTML templates.
- static/css/ — stylesheet files.
- static/js/ — client-side JavaScript when needed.
- .env — local secrets; do not commit.
- requirements.txt — Python dependencies.

Data Flow

- User enters fitness information.
- Jinja2 form sends the request to FastAPI.
- FastAPI validates and normalizes the data.
- Application builds a controlled prompt.
- Gemini generates the plan.
- Backend validates/cleans the returned content as appropriate.
- Result is passed to a Jinja2 template.
- Browser displays the final plan.

UI Pages

- Home/input page.
- Plan-generation or processing state if implemented.
- Generated-plan/result page.
- Error or validation feedback state.

Design Principles

- Keep API credentials in environment variables.
- Keep AI integration separate from route definitions where practical.
- Use templates for presentation and Python modules for application logic.
- Use clear naming and small modules so the project is easy to demonstrate and maintain.